

# Chapter 0: How I Approach an ML System Design Case

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The framework, explanations and diagrams are my own restatement. Notes marked **Beyond the book** are material I added.

Every case in this handbook, whether feed ranking, ad clicks, friend suggestions or content moderation, follows the same arc: pin down what the product needs, choose how to measure it, build the simplest system that could work, and add complexity only where a measurement shows the simple system falls short. This chapter is that arc as a reusable procedure.

## 1. The core principle: minimum sufficient design

**Choose the smallest architecture that satisfies the current requirements. Add a component only when it fixes a demonstrated quality, latency, scale or operational problem.**

### Minimum sufficient design



A component belongs in the design only when it traces back to a requirement or a measured limitation.

Requirement → design decision → expected benefit → metric or evidence

Figure 1: Minimum sufficient design

This isn't minimalism for its own sake. Every component you add is something to train, serve, monitor, debug and explain. A two-stage retrieval + ranking system is the right answer at  $10^9$  items. At  $10^4$  items it's overhead, because you can score everything. The interviewer (or the design review) wants to see that you know *which* problem each component solves.

## 2. The design order

| # | Step         | The question I answer                                                                        |
|---|--------------|----------------------------------------------------------------------------------------------|
| 1 | <b>Scope</b> | What exactly is the input, the model's output and the product's action? What's out of scope? |

| #  | Step                             | The question I answer                                                                                                 |
|----|----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 2  | <b>Requirements</b>              | Scale (users, items, QPS), latency budget, freshness, languages and modalities, cost, fairness and safety constraints |
| 3  | <b>Service decision</b>          | Which decision does the score feed: rank, filter, threshold, route to a human?                                        |
| 4  | <b>Metrics</b>                   | One primary product metric, guardrails, and offline metrics that predict them                                         |
| 5  | <b>Data and labels</b>           | What is one training example? Where do labels come from, how noisy are they, when do they arrive?                     |
| 6  | <b>Baseline</b>                  | The simplest credible model: heuristics, logistic regression, popularity, collaborative filtering                     |
| 7  | <b>Richer alternatives</b>       | What specific limitation of the baseline does each fix?                                                               |
| 8  | <b>Features and training</b>     | Feature families, freshness, point-in-time correctness, loss, sampling, imbalance                                     |
| 9  | <b>Serving and scale</b>         | Online vs precomputed, candidate generation, caching, ANN, latency budget per stage                                   |
| 10 | <b>Deployment and monitoring</b> | Offline gate → shadow → canary → A/B → ramp; drift, retraining triggers, rollback                                     |
| 11 | <b>Decision record</b>           | What I chose, and what measured result would make me switch                                                           |

### Back-of-envelope scale

Numbers turn “it’s big” into architecture. For example, with 500M daily active users making about 10 feed requests a day:

$500\text{e}6 \times 10 / 86,400 \text{ s} \approx 58,000 \text{ requests/s}$  on average → plan for ~2–3× at peak

If each request ranks 1,000 candidates, that’s about  $6 \times 10^7$  model scores per second. That’s why the final ranker only ever sees a small candidate set, and why retrieval uses precomputed embeddings and ANN search.

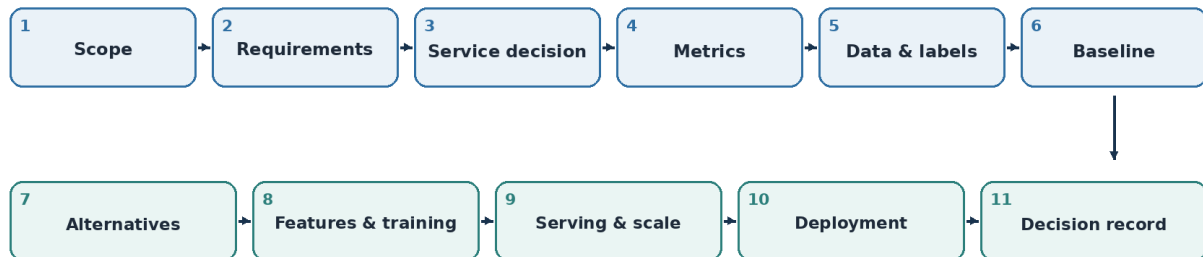
### 3. Three lenses on alternatives

For each design area I consider three perspectives:

- **Easy to implement:** a credible baseline for fast validation, debugging and cheap serving. It’s also the yardstick every other option has to beat.

## Recurring order of an ML system design case

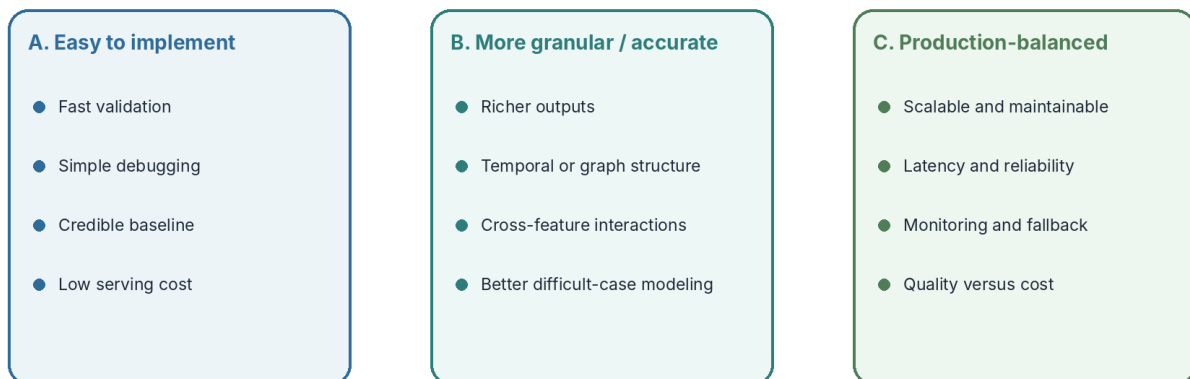
A checklist for completeness, not a rigid script.



Move a section earlier only when a case-specific constraint changes the architecture.

Figure 2: Design order

## Three recurring perspectives for alternatives



These are decision lenses, not a requirement to invent exactly three separate models.

Figure 3: Alternatives lens

- **More granular or accurate:** a richer option aimed at a specific gap: temporal dynamics, graph structure, multiple modalities, feature interactions, per-behavior outputs.
- **Production-balanced:** the option that best trades off quality, latency, scale, reliability, cost and maintainability. This is often a *combination*: a cheap stage to narrow the candidates, then a rich model on the few that remain.

These are ways of looking at the problem, not a rule to always build three models.

## 4. How I compare alternatives

### How to compare an alternative

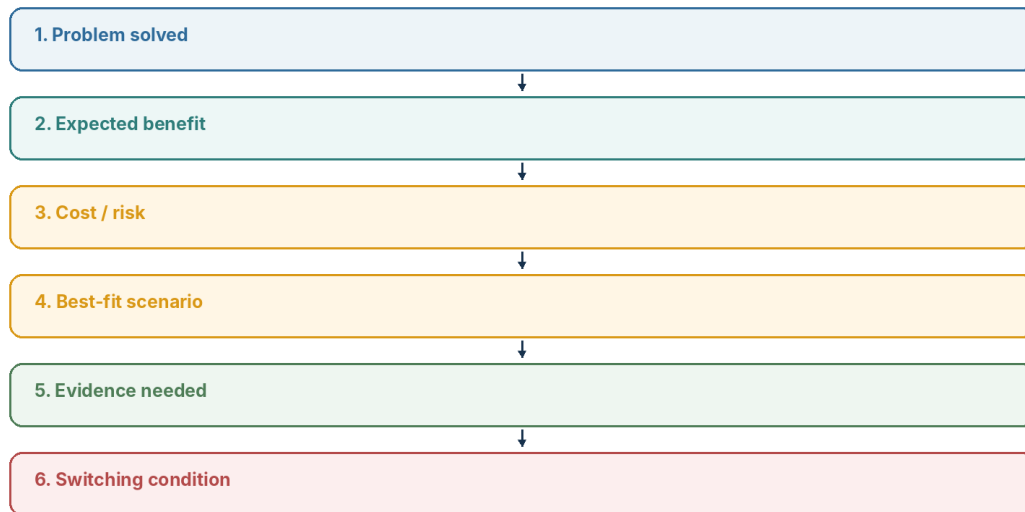


Figure 4: Compare alternatives

For every option I write down six things:

1. **the problem it solves**, e.g. “the LR baseline can’t learn user × ad-category interactions”;
2. **the expected benefit**, e.g. “+0.5-1% relative normalized entropy”;
3. **the cost or risk**: latency, training cost, operational burden, interpretability;
4. **where it fits best**: data size, traffic, freshness needs;
5. **the evidence needed**: an offline metric on a time-based holdout, then an online test;
6. **the switch condition**, stated so it can be measured, e.g. “adopt DCN if it improves offline NE by  $\geq 0.5\%$  relative, holds p99 latency under 20 ms, and wins a two-week A/B test on revenue without raising hide rate.”

Point 6 is what separates a design decision from a preference.

## 5. Model output ≠ service action

Keep the model’s granular predictions separate from the product’s decision:

```

model:  P(like), P(comment), P(hide), ...    or    P(violence), P(nudity), ...
policy: score =  $\sum w_i \cdot p_i$ , thresholds, caps, business rules, overrides
action: rank / show / demote / send to review / remove
  
```

This separation lets you change product policy (weights, thresholds, frequency caps) without retraining. It keeps the model’s outputs meaningful as probabilities. And it gives you per-

## Separate the model output from the service action



### Examples

- category scores → harmful score → keep / review / remove
- candidate scores → ranking policy → ordered recommendation list
- event probability → threshold policy → alert / no alert

Figure 5: Model and service separation

component diagnostics when something goes wrong. It only works if those probabilities are **calibrated**: a weighted sum of miscalibrated probabilities is not a meaningful utility.

## 6. Staged production design

Simple and rich models usually coexist as stages:

millions of items → cheap retrieval (ANN, graph, rules) → ~1,000 candidates  
→ rich ranker → ~50 items → policy / diversity / business rules → shown

Each stage has its own metric: recall@K for retrieval, nDCG or precision@K for ranking, and the product metric at the end.

## 7. Serving and deployment

**Serving mode**, chosen by freshness and latency:

| Mode                | Use when                                                       | Example                                         |
|---------------------|----------------------------------------------------------------|-------------------------------------------------|
| online              | the input only exists at request time, or freshness matters    | feed ranking with the latest interactions       |
| precomputed (batch) | inputs change slowly and the set of users/items is predictable | nightly friend suggestions for active users     |
| hybrid              | stable parts precomputed, a light fresh re-rank online         | cached candidates re-ranked with fresh features |

**Rollout order.** Each stage answers a different question, so I use them in sequence:

1. **Offline evaluation** on a time-based holdout: *is it better on data it hasn't seen?*

## A production-balanced design can combine alternatives

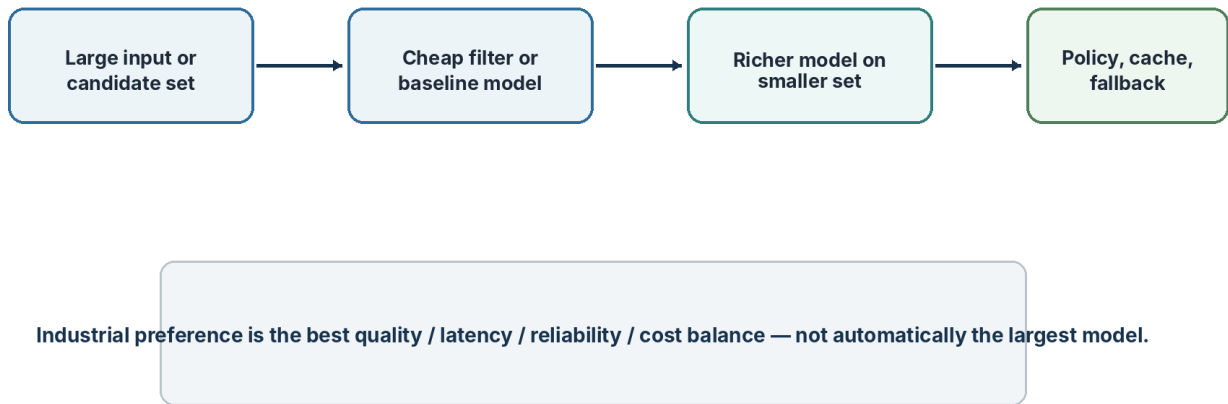
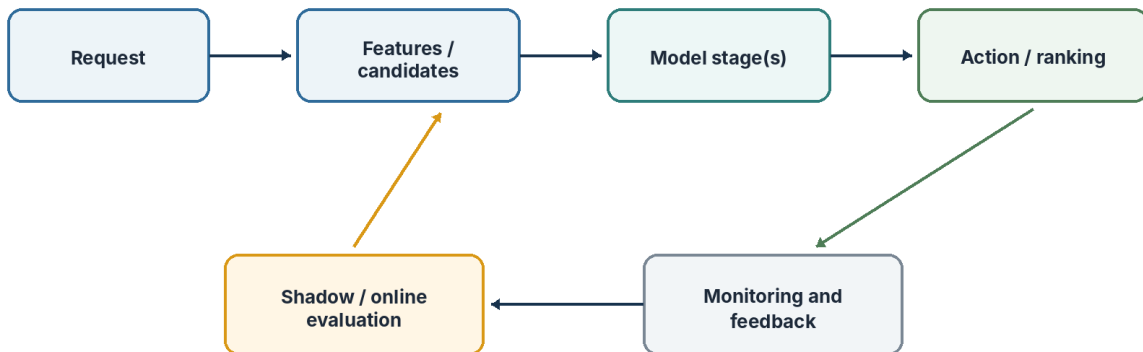


Figure 6: Staged production design

## Serving, validation, and monitoring



Use the lowest-risk validation method that can answer the question.

Figure 7: Serving and deployment

2. **Shadow deployment:** the new model scores live traffic without acting. *Does it run correctly (latency, errors, score distributions)?*
3. **Canary:** real actions for a small slice of traffic. *Is it healthy on real users?*
4. **A/B test:** randomized treatment vs control, e.g. 5% / 95%. *Does it causally improve the product metric without hurting guardrails?*
5. **Ramp to 100%,** with rollback available at every step.

Interleaving is a faster, more sensitive alternative to A/B for comparing *rankers*: it mixes two rankings in one list and counts which one's items get clicked. A/B is still needed for product-level effects.

## 8. Closing statement

"I'd start with the smallest system that satisfies the core requirements, establish a measurable baseline, and add complexity only when a specific product, quality, latency or scale limitation justifies it, stating in advance the measured result that would make me switch."

## 9. Questions and answers

### Q: How do I budget a 45-minute interview?

About 5 minutes on scope and requirements, 5 on metrics, 5 on data and labels, 15 on the model (baseline → richer option), 10 on serving and scale, and 5 on deployment and monitoring. Say the plan out loud at the start so the interviewer can redirect you.

### Q: How do I pick the primary metric when the product has several goals?

Choose the one closest to long-term user value that a two-week experiment can still move (e.g. meaningful interactions or accepted connections, not raw clicks). Make the others guardrails with explicit "must not drop more than X%" limits.

### Q: When is a heuristic baseline acceptable in production?

When it meets the requirement and the ML gain wouldn't pay for its cost. Popularity plus recency, or a rules-based filter, is sometimes the right launch system, and it becomes the control arm of every later experiment.

### Q: What must every design mention, whatever the case?

Point-in-time features (no leakage), a time-based validation split, a calibration check if scores are combined or thresholded, a rollout plan, and monitoring with a defined retraining or rollback trigger.

---